

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

字符串处理与进制转换

String Processing and Base Conversion

作者：Wu Youfan
School of Computing and Data Science, Xiamen University Malaysia

2026 年 10 月 30 日

目录

1. 字符与字符串基础
2. 字符串读入
3. 字符串遍历与统计
4. 回文串判断
5. 进制转换
6. 字符串解析

目录

⑦ 经典例题

⑧ 总结

目录

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

1. 字符与字符串基础
2. 字符串读入
3. 字符串遍历与统计
4. 回文串判断
5. 进制转换
6. 字符串解析

目录

7 经典例题

8 总结

什么是字符

字符：一个字母、数字或符号，用 `char` 类型存储。

本质是整数

- 每个字符背后都有一个编号，叫做 ASCII 码
- `'A'` 是 65，`'a'` 是 97，`'0'` 是 48
- 字符相加不是文字拼接，而是对应编号相加

三条常用的等距规律

- 大小写差 32：`'a' - 'A' == 32`
- 数字连续：`'0'` 到 `'9'` 依次加 1
- 字母连续：`'a'` 到 `'z'` 依次加 1

字符本质上就是 0 到 255 的小整数

字符判断与大小写转换

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

库函数版：需包含头文件 ctype.h

- `isalpha(c)`：是否为字母
- `isdigit(c)`：是否为数字
- `isupper(c)`、`islower(c)`：是否为大写/小写
- `isspace(c)`：是否为空白（空格、回车、Tab）
- `toupper(c)`、`tolower(c)`：转大小写

手写版：不依赖库，条件判断更直观

- `'0' <= c && c <= '9'` 判断数字
- `'a' <= c && c <= 'z'` 判断小写字母
- 大写转小写：`c - 'A' + 'a'`
- 数字字符转整数：`c - '0'`

库函数更方便，手写版更好记

字符串的存储

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

C 风格字符串：一个以 `'\0'` 结尾的字符数组。

- `char s[105];` 最多存 104 个字符，末尾一定是 `'\0'`
- `'\0'` 是字符串结束标记，值为 0，不显示
- 长度是实际字符数，用 `strlen(s)` 得到

关键细节

- 数组开大一号，给 `'\0'` 留位置
- 手写循环遍历：`for (int i = 0; s[i]; i++)`
- 若循环条件是 `i < strlen(s)`，每次会重新算长度，效率低

开 N+1 个位置，用 s[i] 是否为 0 判断结束

目录

1 字符与字符串基础

2 字符串读入

3 字符串遍历与统计

4 回文串判断

5 进制转换

6 字符串解析

目录

7 经典例题

8 总结

三种常用读入方式

方式一： `scanf("%s", s)` 读一个单词

- 遇空格、回车、Tab 就停
- 不含空格时最省事，输入 `hello world` 只会读到 `hello`

方式二： `fgets(s, MAXN, stdin)` 读一整行

- 包含空格，末尾会带一个换行符
- 读入后通常手动去掉末尾的 `'\n'`
- 安全版，替代已废弃的 `gets`

方式三： `cin.getline(s, MAXN)` 或 `getline(cin, str)` 读一整行

- 同样读一整行，不保留换行符

读单词用 scanf，读整行用 fgets

读入的常见坑

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

坑一：换行符残留

- 先用 `scanf("%d", &n)` 读数字，接下来的读整行会读到空串
- 原因：数字后面的回车符被下一行读走了
- 解决：中间加一句 `getchar();` 吃掉换行符

坑二：fgets 保留了换行

- `fgets` 读到的字符串末尾有 `'\n'`
- 处理：读入后判断末尾是否为 `'\n'`，是就改成 `'\0'`
- 或直接不处理，让循环遇 `'\n'` 时特殊判断

读数字之后、读整行之前，先吃换行符

目录

1 字符与字符串基础

2 字符串读入

3 字符串遍历与统计

4 回文串判断

5 进制转换

6 字符串解析

目录

7 经典例题

8 总结

例题：统计字母数字空格

题目：输入一行字符串（含空格），分别统计其中字母、数字、空格的数量，输出三个数。

示例

- 输入 `abc 123 def`
- 输出 `6 3 2`（6 个字母，3 个数字，2 个空格）
- 输入 `Hello World`
- 输出 `10 0 1`

统计字母数字空格：解法

核心思路：遍历字符串的每一个字符，用类型判断函数分辨它属于哪一类，对应计数器加 1。

关键步骤

- 用 `fgets` 读整行，因为字符串里可能有空格
- 三个计数器 `letters`、`digits`、`spaces` 都置 0
- 循环 `for (int i = 0; s[i]; i++)`，遇 `'\0'` 结束
- 用 `isalpha`、`isdigit` 判断类型，对应计数器加 1
- 用 `' '` 判断空格

遍历一遍，每个字符分个类

统计字母数字空格：代码

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```c
#include <stdio.h>
#include <ctype.h>
#include <string.h>

int main() {
    char s[1005];
    fgets(s, sizeof(s), stdin);        // 读整行
    int letters = 0, digits = 0, spaces = 0;
    for (int i = 0; s[i]; i++) {       // 遇 '\0' 结束
        if (isalpha(s[i]))      letters++;
        else if (isdigit(s[i])) digits++;
        else if (s[i] == ' ')   spaces++;
    }
    printf("%d %d %d\n", letters, digits, spaces);
    return 0;
}
```

目录

1 字符与字符串基础

2 字符串读入

3 字符串遍历与统计

4 回文串判断

5 进制转换

6 字符串解析

目录

7 经典例题

8 总结

例题：判断回文串

题目：输入一个字符串，判断它是不是回文串。回文串指从左往右读和从右往左读完全相同。

示例

- 输入 `level`，输出 `yes`
- 输入 `hello`，输出 `no`
- 输入 `abba`，输出 `yes`

判断回文串：解法

核心思路：双指针从两端往中间走，对应字符都相同才是回文。

关键步骤

- 计算长度 `len = strlen(s)`
- 左指针 `i = 0`，右指针 `j = len - 1`
- 循环条件 `i < j`
- 每轮比较 `s[i]` 和 `s[j]`，不同就返回 false
- 相同则 `i++`、`j--`，继续往中间靠
- 循环结束没发现不同，就是回文

两端向中间夹，全对称就是回文

判断回文串：代码


```c
#include <stdio.h>
#include <string.h>

int main() {
    char s[1005];
    scanf("%s", s);
    int len = strlen(s);
    int i = 0, j = len - 1;
    int is_pal = 1;
    while (i < j) {
        if (s[i] != s[j]) { is_pal = 0; break; }
        i++;
        j--;
    }
    printf("%s\n", is_pal ? "yes" : "no");
    return 0;
}
```

目录

1 字符与字符串基础

2 字符串读入

3 字符串遍历与统计

4 回文串判断

5 进制转换

6 字符串解析



目录

7 经典例题

8 总结

例题：十进制转 k 进制

题目：给定一个正整数 n 和一个进制 k $(2 \le k \le 16)$，把 n 转成 k 进制输出。大于 9 的数位用大写字母 A 到 F 表示。

示例

- $n = 10$，$k = 2$，输出 `1010`
- $n = 255$，$k = 16$，输出 `FF`
- $n = 0$，$k = 2$，输出 `0`

十进制转 k 进制：解法

核心思路：除 k 取余，把每次的余数存下来，最后倒序输出。

关键步骤

- 特判 `n == 0`，直接输出 0
- 用一个数组 `digits` 记录余数，计数器 `cnt` 从 0 开始
- 循环 `while (n > 0)`：余数 `n % k` 存入数组，`n /= k`
- 倒序输出：`for (int i = cnt - 1; i >= 0; i--)`
- 余数 `< 10` 输出数字，否则输出 `digits[i] - 10 + 'A'`

除 k 取余，余数倒序就是答案

十进制转 k 进制：代码


```
1  void convert(int n, int k) {
2      if (n == 0) { printf("0\n"); return; }
3      int digits[50], cnt = 0;
4      while (n > 0) {
5          digits[cnt++] = n % k;    // 取余
6          n /= k;                   // 除 k
7      }
8      for (int i = cnt - 1; i >= 0; i--) {   // 倒序输出
9          if (digits[i] < 10)
10             printf("%d", digits[i]);
11         else
12             printf("%c", digits[i] - 10 + 'A');
13     }
14     printf("\n");
15 }
```

例题：k 进制转十进制

题目：给定一个 k 进制字符串 s $(2 \le k \le 16)$，转成十进制整数输出。数位 A 到 F 对应 10 到 15。

示例

- 输入字符串 `1010`，$k = 2$，输出 10
- 输入字符串 `FF`，$k = 16$，输出 255
- 输入字符串 `777`，$k = 8$，输出 511

k 进制转十进制：解法

核心思路：从左往右遍历，每读一位就把结果乘 k 再加这一位的值，这叫“乘基加位”。

关键步骤

- 结果 `result = 0`
- 遍历每一位字符 `s[i]`
- 若是数字字符，位数 `d = s[i] - '0'`
- 若是字母，位数 `d = s[i] - 'A' + 10`
- 更新 `result = result * k + d`
- 循环结束，`result` 就是答案

边读边乘，累积出十进制值

k 进制转十进制：代码


```
int toDecimal(char* s, int k) {
    int result = 0;
    for (int i = 0; s[i]; i++) {
        int d;
        if (s[i] >= '0' && s[i] <= '9')
            d = s[i] - '0';
        else
            d = s[i] - 'A' + 10;
        result = result * k + d;
    }
    return result;
}
```

目录

1 字符与字符串基础

2 字符串读入

3 字符串遍历与统计

4 回文串判断

5 进制转换

6 字符串解析

目录

7 经典例题

8 总结

例题：提取字符串中所有数字

题目：给定一个字符串，提取其中连续出现的所有数字，按顺序输出，每个数字之间用一个空格分隔。

示例

- 输入 `abc123def45`，输出 `123 45`
- 输入 `a1b22c333`，输出 `1 22 333`
- 输入 `no_digits_here`，无输出

提取所有数字：解法

核心思路：从左往右扫描，遇到数字就开始累积，遇到非数字就结束一段。

关键步骤

- 用指针 `i` 扫描整个字符串
- 若 `s[i]` 是数字，记下起点 `start = i`
- 继续往后走，直到遇到非数字或结束，记 `end = i`
- 用 `for (j = start; j < end; j++)` 把这一段拼成整数
- 输出这个数，把 `i` 跳到 `end` 继续
- 若 `s[i]` 不是数字，`i` 直接加 1

一段一段抓，非数字就切段

提取所有数字：代码


```
1  void extractNumbers(char* s) {
2      int i = 0;
3      while (s[i]) {
4          if (s[i] >= '0' && s[i] <= '9') {
5              int num = 0;
6              while (s[i] >= '0' && s[i] <= '9') {
7                  num = num * 10 + (s[i] - '0');
8                  i++;
9              }
10             printf("%d ", num);
11         } else {
12             i++;
13         }
14     }
15     printf("\n");
16 }
```

目录

1 字符与字符串基础

2 字符串读入

3 字符串遍历与统计

4 回文串判断

5 进制转换

6 字符串解析

目录

❼ 经典例题

❽ 总结

例题：高精度加法

题目：给定两个超长正整数（可能超过 `long long`），输出它们的和。每个数的长度不超过 1000 位。

示例

- 输入 `123` 和 `456`，输出 `579`
- 输入 `999` 和 `1`，输出 `1000`
- 输入 `12345678901234567890` 和 `98765432109876543210`，输出 `111111111011111111100`

高精度加法：解法

核心思路：把字符串倒序存进两个整数数组，然后从个位开始逐位相加，处理进位，最后倒序输出。

关键步骤

- 读入两个字符串 `sa`、`sb`
- 倒序存到 `a[]`、`b[]`：`a[i] = sa[len-1-i] - '0'`
- 长度 `len = max(lena, lenb)`
- 逐位相加：`c[i] += a[i] + b[i]`，进位处理 `c[i+1] += c[i] / 10`、`c[i] %= 10`
- 最高位若还有进位，长度加 1
- 倒序输出 `c[]`

倒序存、逐位加、进位传、倒序出

高精度加法：代码


```
1  char sa[1005], sb[1005];
2  int a[1005], b[1005], c[1005];
3
4  int main() {
5      scanf("%s %s", sa, sb);
6      int la = strlen(sa), lb = strlen(sb);
7      // 倒序存到数组
8      for (int i = 0; i < la; i++) a[i] = sa[la-1-i] - '0';
9      for (int i = 0; i < lb; i++) b[i] = sb[lb-1-i] - '0';
10     // 逐位相加处理进位
11     int len = la > lb ? la : lb;
12     for (int i = 0; i < len; i++) {
13         c[i] += a[i] + b[i];
14         c[i+1] += c[i] / 10;
15         c[i] %= 10;
16     }
17     // 最高位有进位则长度加 1
18     if (c[len] > 0) len++;
```

高精度加法：代码


```
19      // 倒序输出
20      for (int i = len - 1; i >= 0; i--) printf("%d", c[i]);
21      printf("\n");
22      return 0;
23  }
```

例题：括号匹配

题目：给定一个只含小括号、中括号、大括号的字符串，判断括号是否合法匹配。每个左括号必须有一个同类型的右括号，且嵌套顺序正确。

示例

- 输入 `()`、`()[]{}`、`{[()]}`，都合法
- 输入 `(]`、`([)]`、`(`，都不合法

括号匹配：解法

核心思路：用栈（数组模拟）保存左括号，遇到右括号就尝试匹配栈顶，最后栈空才算合法。

关键步骤

- 定义一个字符数组 `stack` 作为栈，`top = 0`
- 遍历字符串每个字符
- 遇左括号 `(`、`[`、左大括号：压入栈，`stack[top++] = c`
- 遇右括号：若栈空则不合法；否则检查栈顶是否同类
- 若同类则弹出栈顶 `top--`，否则不合法
- 循环结束若 `top == 0` 则合法，否则不合法

左括号入栈，右括号对比栈顶

括号匹配：代码

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```
1  int is_match(char l, char r) {
2      return (l == '(' && r == ')')
3          || (l == '[' && r == ']')
4          || (l == '{' && r == '}');
5  }
6
7  int check(char* s) {
8      char stack[1005];
9      int top = 0;
10     for (int i = 0; s[i]; i++) {
11         char c = s[i];
12         if (c == '(' || c == '[' || c == '{') {
13             stack[top++] = c;          // 左括号入栈
14         } else {
15             if (top == 0) return 0;    // 栈空遇到右括号
16             if (!is_match(stack[top-1], c)) return 0;
17             top--;                     // 匹配成功出栈
18         }
```

括号匹配：代码


```
19      }
20      return top == 0;                    // 栈空才算合法
21  }
```

例题：化学分子式质量计算

题目：给定一个化学分子式（只含元素符号和下标数字），计算其相对分子质量。常见元素质量为 H=1、C=12、N=14、O=16、Na=23、Cl=35.5。下标数字可能有多位，省略时表示 1。

示例

- `H2O`：2 个 H + 1 个 O，总质量 `2*1 + 16 = 18`
- `CaCO3`：1 个 Ca + 1 个 C + 3 个 O，总质量 `40 + 12 + 3*16 = 100`
- `NaCl`：1 个 Na + 1 个 Cl，总质量 `23 + 35.5 = 58.5`

化学分子式质量计算：解法

核心思路：从左往右扫，每次识别出“一个元素符号 + 可能的数字”，查表拿质量，乘以个数累加。

关键步骤

- 准备查表函数 `get_weight(sym)`：返回元素相对质量
- 识别元素符号：读一个大写字母，看下一位是不是小写字母，是就一起读
- 读下标数字：读一位或多位数字，没有则默认为 1
- 累加：`total += get_weight(sym) * cnt`
- 指针移动到这段之后，继续下一轮

关键点：元素符号可能是 1 个字符（如 H）或 2 个字符（如 Na），先读大写、再看小写是关键。

识别元素 + 识别下标 + 累加

化学分子式质量计算：代码（识别）

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```c
double get_weight(char* sym) {
    if (strcmp(sym, "H") == 0)  return 1.0;
    if (strcmp(sym, "C") == 0)  return 12.0;
    if (strcmp(sym, "N") == 0)  return 14.0;
    if (strcmp(sym, "O") == 0)  return 16.0;
    if (strcmp(sym, "Na") == 0) return 23.0;
    if (strcmp(sym, "Cl") == 0) return 35.5;
    if (strcmp(sym, "Ca") == 0) return 40.0;
    return 0;   // 遇到未收录元素返回 0
}
```

用 strcmp 逐项比较元素符号，返回相对质量

化学分子式质量计算：代码（主循环）


```
double calc(char* s) {
    double total = 0;
    int i = 0;
    while (s[i]) {
        char sym[3] = {0};
        sym[0] = s[i++];                    // 读大写字母
        if (s[i] >= 'a' && s[i] <= 'z')     // 看是否有小写
            sym[1] = s[i++];
        int cnt = 0;
        while (s[i] >= '0' && s[i] <= '9') {
            cnt = cnt * 10 + (s[i] - '0');
            i++;
        }
        if (cnt == 0) cnt = 1;              // 没有下标默认为 1
        total += get_weight(sym) * cnt;
    }
    return total;
}
```

目录

1 字符与字符串基础

2 字符串读入

3 字符串遍历与统计

4 回文串判断

5 进制转换

6 字符串解析

目录

⑦ 经典例题

⑧ 总结

总结（一）：字符与字符串速查

用途	核心要点
判断数字	检查是否在字符 0 到 9 的区间内
判断字母	检查是否在字符 a 到 z 或 A 到 Z 的区间内
大小写转换	大写字母加 32 或小写字母减 32
字符转整数	字符直接减去字符 0
长度获取	使用 strlen 函数
读整行	使用 fgets 读取到标准输入
读单词	使用 scanf 配合%s
遍历循环	循环条件写 s[i] 非零即可

字符即整数，判断靠区间

总结（二）：进制转换速查

方向	核心操作
十转 k	循环取 n 对 k 的余数，然后 n 除以 k
输出	倒序输出余数，大于 9 转大写 A 到 F
k 转十	结果乘以 k 加上当前位数的值
位数解析	数字字符减 0，字母减 A 再加 10

除基取余倒序写，乘基加位顺着读

总结（三）：常见易错点

读入类

- 数字后读字符串读到空串，忘记吃掉换行符
- `fgets` 保留 `'n'` 没处理，影响后续比较
- 字符数组开得太小，加 `'0'` 就溢出

逻辑类

- 进制转换时忘记特判 `n == 0`
- 括号匹配时未检查栈空，遇右括号直接越界
- 高精度加法忘记处理最高位进位，结果少一位
- 化学分子式未区分单字符元素和双字符元素，如 `Na` 被错读为 `N + a`

读入看换行，逻辑看边界

附录：常见陷阱与调试技巧

调试字符串

- 打印长度：`printf("len = %d\n", (int)strlen(s));`
- 逐字符打印：`printf("[%c]", s[i]);`，能看出是否有多余空格或换行
- 十六进制打印：`printf("%d ", s[i]);`，看 ASCII 码定位异常字符

边界测试

- 空字符串、单字符字符串都要试
- 进制转换试 `n = 0`、`n = 1` 等极端值
- 高精度加法试两个长度不同的数、有连续进位的数（如 $999 + 1$）

所有边界用小数据过一遍，稳过一半测点